

Lecture 13

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Rootfinding

Goals:

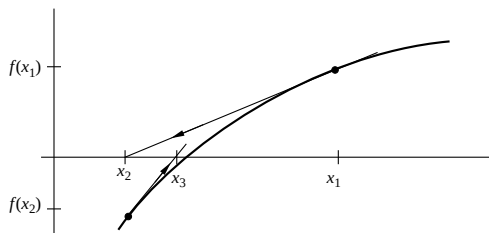
- Find roots to equations
 - Compare usability of different methods
 - Compare convergence properties of different methods
- 1 bracketing methods
 - 2 Bisection Method
 - 3 **Newton's Method**
 - 4 **Secant Method**

Performing Division

- Ever wondered how a computer process performs division?
- “Long” division requires lookup, subtraction, shifts
- Generates one digit at a time. Can we do better?

To answer this, we need to look at faster methods than bisection

Newton's Method



For a current guess x_k , use $f(x_k)$ and the slope $f'(x_k)$ to predict where $f(x)$ crosses the x axis.

Newton's Method

Expand $f(x)$ in Taylor Series around x_k

$$f(x_k + \Delta x) = f(x_k) + \Delta x \left. \frac{df}{dx} \right|_{x_k} + \frac{(\Delta x)^2}{2} \left. \frac{d^2f}{dx^2} \right|_{x_k} + \dots$$

Substitute $\Delta x = x_{k+1} - x_k$
and neglect 2nd order terms to get

$$f(x_{k+1}) \approx f(x_k) + (x_{k+1} - x_k) f'(x_k)$$

where

$$f'(x_k) = \left. \frac{df}{dx} \right|_{x_k}$$

Newton's Method

Goal is to find x such that $f(x) = 0$.

Set $f(x_{k+1}) = 0$ and solve for x_{k+1}

$$0 = f(x_k) + (x_{k+1} - x_k)f'(x_k)$$

or, solving for x_{k+1}

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Newton's Method Algorithm

```
1 initialize:  $x_1 = \dots$   
2 for  $k = 2, 3, \dots$   
3    $x_k = x_{k-1} - f(x_{k-1})/f'(x_{k-1})$   
4   if converged, stop  
5 end
```

Newton's Method Example

Solve:

$$x - x^{1/3} - 2 = 0$$

First derivative is

$$f'(x) = 1 - \frac{1}{3}x^{-2/3}$$

The iteration formula is

$$x_{k+1} = x_k - \frac{x_k - x_k^{1/3} - 2}{1 - \frac{1}{3}x_k^{-2/3}}$$

Newton's Method Example

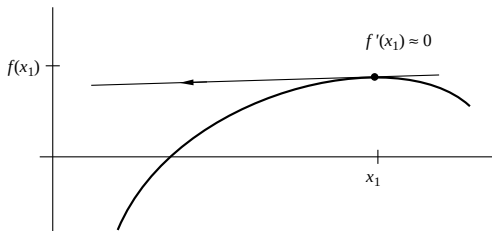
$$x_{k+1} = x_k - \frac{x_k - x_k^{1/3} - 2}{1 - \frac{1}{3}x_k^{-2/3}}$$

k	x_k	$f'(x_k)$	$f(x)$
0	3	0.83975005	-0.44224957
1	3.52664429	0.85612976	0.00450679
2	3.52138015	0.85598641	3.771×10^{-7}
3	3.52137971	0.85598640	2.664×10^{-15}
4	3.52137971	0.85598640	0.0

Conclusion

- Newton's method converges *much* more quickly than bisection
- Newton's method requires an analytical formula for $f'(x)$
- The algorithm is simple as long as $f'(x)$ is available.
- Iterations are not guaranteed to stay inside an ordinal bracket.

Divergence of Newton's Method



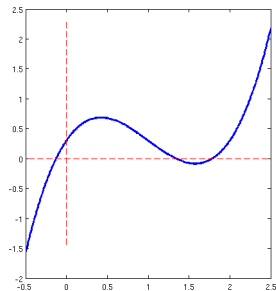
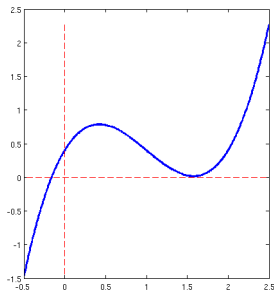
Since

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

the new guess, x_{k+1} , will be far from the old guess whenever $f'(x_k) \approx 0$

Divergence of Newton's Method

Can you guess?



Newton's Method: Convergence

Recall

Convergence of a method is said to be of order r if there is a constant $C > 0$ such that

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^r} = C$$

Newton's method is of order 2 (quadratic) when $f'(x_*) \neq 0$. For ξ between x_k

and x_*

$$f(x_*) = f(x_k) + (x_* - x_k)f'(x_k) + \frac{1}{2}(x_* - x_k)^2 f''(\xi) = 0$$

So

$$\frac{f(x_k)}{f'(x_k)} + x_* - x_k + (x_* - x_k)^2 \frac{f''(\xi)}{f'(x_k)} = 0$$

Then

$$x_* - x_{k+1} + (x_* - x_k)^2 \frac{f''(\xi)}{f'(x_k)} = 0$$

Thus

$$\frac{|x_* - x_{k+1}|}{|x_* - x_k|^2} = \frac{f''(\xi)}{f'(x_k)}$$

Reciprocal Approximation

- Consider the task of computing $1/q$ for some q without using division.
- We can write this as: find the root x of $f(x) = 1/(xq) - 1 = 0$.
- What is Newton's Method for this?
- $f'(x) = -1/(x^2q)$. Thus

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

or

$$x_{n+1} = x_n - \frac{1/(x_n q) - 1}{-1/(x_n^2 q)}$$

Reciprocal Approximation

- Consider the task of computing $1/q$ for some q without using division.
- We can write this as: find the root x of $f(x) = 1/(xq) - 1 = 0$.
- What is Newton's Method for this?
- $f'(x) = -1/(x^2q)$. Thus

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

or

$$x_{n+1} = x_n - \frac{1/(x_n q) - 1}{-1/(x_n^2 q)} \frac{x_n^2 q}{x_n^2 q}$$

$$x_{n+1} = x_n + x_n - x_n^2 q = 2x_n - x_n^2 q$$

Example: Compute $1/3$

- Find the bracket:

- $1/2 > 1/3 > 1/4$

① $x_0 = 1/4$

② $x_1 = 2x_0 - x_0^2q = 1/2 - 3/16 = 5/16$

③ $x_2 = 2 \times 5/2^4 - 3 \times 25/2^8 = (160 - 75)/2^8 = 85/2^8$

④ $x_3 = 2 \times 85/2^8 - 3 \times 85^2/2^{16} = 21845/2^{16}$

In 3 steps, computed 16 bits in $1/3$

How many binary digits are computed in the next step?

Secant Method

Given

x_k = current guess at the root

x_{k-1} = previous guess at the root

Approximate the first derivative with

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

Substitute approximate $f'(x_k)$ into formula for Newton's method

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

to get

$$x_{k+1} = x_k - f(x_k) \left[\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \right]$$

Secant Method

Two versions of this formula are (equivalent in exact math)

$$x_{k+1} = x_k - f(x_k) \left[\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \right] \quad (\star)$$

and

$$x_{k+1} = \frac{f(x_k)x_{k-1} - f(x_{k-1})x_k}{f(x_k) - f(x_{k-1})} \quad (\star\star)$$

Equation $(\star)$ is better since it is of the form $x_{k+1} = x_k + \Delta$. Even if Δ is inaccurate the change in the estimate of the root will be small at convergence because $f(x_k)$ will also be small.

Equation $(\star\star)$ is susceptible to catastrophic cancellation:

- $f(x_k) \rightarrow f(x_{k-1})$ as convergence approaches, so cancellation error in denominator can be large.
- $|f(x)| \rightarrow 0$ as convergence approaches, so underflow is possible

Secant Algorithm

```
1 initialize:  $x_1 = \dots, x_2 = \dots$   
2 for  $k = 2, 3 \dots$   
3    $x_{k+1} = x_k - f(x_k)(x_k - x_{k-1}) / (f(x_k) - f(x_{k-1}))$   
4   if converged, stop  
5 end
```

Secant Example

Solve:

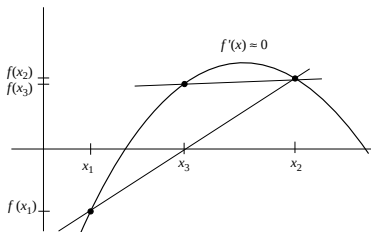
$$x - x^{1/3} - 2 = 0$$

k	x_{k-1}	x_k	$f(x_k)$
0	4	3	-0.44224957
1	3	3.51734262	-0.00345547
2	3.51734262	3.52141665	0.00003163
3	3.52141665	3.52137970	-2.034×10^{-9}
4	3.52137959	3.52137971	-1.332×10^{-15}
5	3.52137971	3.52137971	0.0

Conclusions:

- Converges almost as quickly as Newton's method ($r \approx 1.62$).
- There is no need to compute $f'(x)$.
- The algorithm is simple.
- Two initial guesses are necessary
- Iterations are not guaranteed to stay inside an ordinal bracket.

Divergence of Secant Method



Since

$$x_{k+1} = x_k - f(x_k) \left[\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \right]$$

the new guess, x_{k+1} , will be far from the old guess whenever $f(x_k) \approx f(x_{k-1})$ and $|f'(x)|$ is not small.

Summary

- Plot $f(x)$ before searching for roots
- Bracketing finds coarse interval containing roots and singularities
- Bisection is robust, but converges slowly
- Newton's Method
 - Requires $f(x)$ and $f'(x)$.
 - Iterates are not confined to initial bracket.
 - Converges rapidly ($r = 2$).
 - Diverges if $f'(x) \approx 0$ is encountered.
- Secant Method
 - Uses $f(x)$ values to approximate $f'(x)$.
 - Iterates are not confined to initial bracket.
 - Converges almost as rapidly as Newton's method ($r \approx 1.62$).
 - Diverges if $f'(x) \approx 0$ is encountered.

fzero Function

fzero is a hybrid method that combines bisection, secant and reverse quadratic interpolation

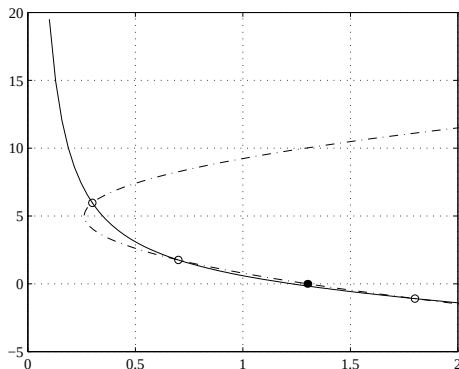
```
1 r = fzero('fun',x0)
2 r = fzero('fun',x0,options)
3 r = fzero('fun',x0,options,arg1,rg2,...)
```

x_0 can be a scalar or a two element vector

- If x_0 is a scalar, fzero tries to create its own bracket.
- If x_0 is a two element vector, fzero uses the vector as a bracket.

Reverse Quadratic Interpolation

Find the point where the x axis intersects the sideways parabola passing through three pairs of $(x, f(x))$ values.



fzero Function

fzero chooses next root as

- Result of reverse quadratic interpolation (RQI) if that result is inside the current bracket.
- Result of secant step if RQI fails, and if the result of secant method is inside the current bracket.
- Result of bisection step if both RQI and secant method fail to produce guesses inside the current bracket.

fzero Function

Optional parameters to control fzero are specified with the optimset function.

Tell fzero to display the results of each step:

```
1 >> options = optimset('Display','iter');  
2 >> x = fzero('myFun',x0,options)
```

Tell fzero to use a relative tolerance of 5×10^{-9} :

```
1 >> options = optimset('TolX',5e-9);  
2 >> x = fzero('myFun',x0,options)
```

Tell fzero to suppress all printed output, and use a relative tolerance of 5×10^{-4} :

```
1 >> options = optimset('Display','off','TolX',5e-4);  
2 >> x = fzero('myFun',x0,options)
```

fzero Function

Allowable options (specified via `optimset`):

Option type	Value	Effect
'Display'	'iter'	Show results of each iteration
	'final'	Show root and original bracket
	'off'	Suppress all print out
'TolX'	tol	Iterate until $ \Delta x < \max[\text{tol}, \text{tol} * a, \text{tol} * b]$ where $\Delta x = (b - a)/2$, and $[a, b]$ is the current bracket.

The default values of 'Display' and 'TolX' are equivalent to

```
options = optimset('Display','iter','TolX',eps)
```

fzero example

Take

$$f(x) = x^{10} - 1$$

```
1 >> f = @(x)x.^10 - 1;  
2 >> options = optimset('display','iter');  
3 >> [x,fx]=fzero(f,0.5,options)
```